

# Multiplication-Free Collocation Method (MFCM)

## Elméleti Összefoglaló és Implementációs Terv

Ez a dokumentum a Multiplication-Free Semi-Discretization (MFSD) koncepciójának kiterjesztését írja le tetszőleges konvergenciarendű, implicit Runge-Kutta (kollokációs) integrátorokra. A módszer célja, hogy a hagyományos Semi-Diszkretizáció (SD) alacsony pontosságát spektrális pontosságra cserélje, miközben megőrzi az MFSD lineáris futási idejét ( $\mathcal{O}(p^1)$ ) és alacsony memóriaigényét.

## 1. Elméleti Összefoglaló

### 1.1. A Koncepció Lényege

A hagyományos spektrális és kollokációs módszerek Késleltetett Differenciálegyenletek (DDE) esetén egy sűrű monodrómia mátrixot építenek fel, amelynek kiszámítása köbös ( $\mathcal{O}(p^3)$ ) időkomplexitással jár. Az **MFCM** (Multiplication-Free Collocation Method) ezzel szemben nem számítja ki explicit módon az átmenetmátrixot. Ehelyett az összes időlépés összes köztes állapotát (Runge-Kutta stádiumok) egyetlen hatalmas, folytonos állapotvektorba ( $\mathbf{q}$ ) fűzi, és az egyenleteket nullára rendezi:  $0 = \mathcal{Q}\mathbf{q}$ .

A trükk a  $\mathcal{Q}$  operátor felépítésében rejlik, amely extrém ritka, sávós blokkmátrix, és diadikus (Kronecker-jellegű) szétbontással áll elő. A stabilitásvizsgálathoz ezt a globális egyenletet bontjuk szét egy egyperiódusos átmenetté.

### 1.2. A Globális Állapotvektor

A redundancia elkerülése végett (hogy a szomszédos blokkok "sarkai" pontosan illeszkedjenek), az  $n$ -edik időlépés állapotvektora  $s$  stádiumú (pl.  $s = 3$ ) RK módszer esetén:

$$v_n = \begin{bmatrix} y_n \\ Y_{1,n} \\ \vdots \\ Y_{s,n} \end{bmatrix}$$

A globális vektor ezek egymásutánja:  $\mathbf{q} = [\dots, v_{n-1}^T, v_n^T, v_{n+1}^T, \dots]^T$ .

### 1.3. A Diadikus Lokális Leképezés ( $\mathcal{M}_n$ )

Egyetlen időlépés leképezése három független komponens diadikus szorzatára/kombinációjára bomlik:

1. **Topológiai Csontváz ( $\mathcal{T}$ ):** Konstans mátrix, ami az egyenletek bal oldalát ( $-y_n + Y_{i,n} = \dots$ ) írja le, és összeköti a lépéseket.
2. **Butcher-tábla ( $\mathcal{B}$ ):** Konstans mátrix, ami az  $a_{ij}$  és  $b_j$  súlyokat tartalmazza.
3. **Rendszermátrixok ( $\mathcal{A}_n$ ):** A  $t_n + c_i h$  időpontokban kiértékelt (és elraktározott) aktuális együtthatómátrixok diagonális blokkja.

#### 1.4. A Késleltetés és a Rendvesztés (Order Reduction) elkerülése

A  $B(t)y(t - \tau)$  tagok visszakeresése kritikus. Ha explicit módon tároltuk az  $Y_{i,n-r}$  múltbeli stádiumokat, egyszerű interpoláció (pl. spline) **NEM** használható, mert tönkreteszi a magas rendű módszerek pontosságát. **Megoldás:** A késleltetett tagok kiszámításához szigorúan a kiválasztott Butcher-táblához tartozó **Folytonos Kiterjesztést (Continuous Extension / Dense Output polinomokat)** kell használni. Ezek a polinomok biztosítják, hogy a visszanyúló  $\mathcal{D}_n$  blokkok az RK módszer eredeti bázisán maradjanak, garantálva a tetszőleges konvergenciarendet.

#### 1.5. A $T$ és $\tau_{max}$ arányának kezelése (Bal és Jobb leképezés)

A Floquet-szorzók kereséséhez az egyperiódusos átmenetet kell meghatározni:  $\Phi_L y_p = \Phi_R y_0$ . A  $\mathcal{Q}$  mátrix felbontása és a vektorok kiegészítése a periódusidő ( $T = p \cdot h$ ) és a maximális időkésés ( $\tau = r \cdot h$ ) arányától függ:

- $p - 1 = r$  (**Nincs átfedés**): A  $\mathcal{Q}$  leképezés pontosan két négyzetes  $\Phi_L$  és  $\Phi_R$  operátorra bomlik. A  $y_0$  és  $y_p$  vektorok mérete megegyezik.
- $p - 1 > r$  (**Kiegészítés / Padding**): A periódusidő hosszabb, mint a késleltetés. A  $y_0$  állapotvektort ki kell egészíteni (nullákkal vagy a rendszer belső állapotait virtuálisan kiterjesztve), hogy a  $\Phi_L$  és  $\Phi_R$  dimenziói négyzetesek maradjanak a stabilitásvizsgálathoz. A ritka/lazy reprezentáció miatt ez a kiegészítés számítási időben "ingyen" van.
- $p - 1 < r$  (**Átfedés / Overlap**): A késleltetés hosszabb, mint a periódus. Ekkor az  $y_0$  és  $y_p$  állapotvektorok átfedésben vannak. A  $\Phi_L$  mátrixba kerülnek a lokális frissítések, a  $\Phi_R$  pedig megkapja az átfedő elemeket, megfelelő identitás blokkokkal kiegészítve, hogy biztosítsa a múlt folytonosságát az iterációban.

## 2. Implementációs Terv (Julia)

A kódolást a meglévő `SemiDiscretizationMethod.jl` logikájára alapozva, de egy teljesen új, moduláris `MFCM` (vagy `SpectralSDM`) modulba érdemes szervezni.

### 2.1. Architektúra és Adatstruktúrák

Használjunk `StaticArrays`-t a lokális  $A(t)$  és  $B(t)$  mátrixokhoz (pl. `SMatrix{D,D,Float64}`).

## Szükséges Structok:

- `RKTableau` : Tartalmazza a  $c$ ,  $a$ ,  $b$  vektorokat/mátrixokat, ÉS a *Continuous Extension* kiértékelő függvényét (vagy súlymátrix generátorát adott tört lépésre). Legyen kompatibilis/átemelhető az `OrdinaryDiffEq.jl` tábláiból.
- `TimeGrid` : Tartalmazza a  $t$  rácspontokat. **Követelmény:** Támogassa a nem egyenletes lépésközt is.
- `SystemMatrices` : "Mátrixok mátrixa". Itt előre kiszámítjuk és tároljuk az  $A_{i,n}$  és  $B_{i,n}$  értékeket az összes  $[t_n + c_i h]$  pontra. (Sűrű struktúra, de mivel `SMatrix`, a memórafoglalása minimális).
- `MonodromyMap` : Egy egyedi típus, amely a `LinearMaps.jl` interfésszel rendelkezik a Lazy operációkhoz, és belsőleg kezeli a padding/overlap logikát.

## 2.2. A Rendszer Definiálása (A felhasználói interfész)

A felhasználónak képesnek kell lennie manuálisan, vagy a `DiffEq` interfészen keresztül megadni a rendszert.

### 1. Konstruktor `DiffEq` alapon (Automatikus extrakció):

```
# A felhasználói RHS
function mathieu_dde_rhs(u, h, p, t)
    δ, ε, b0, a1, T, τ_val = p
    hist = h(p, t - τ_val)
    du1 = u[2]
    du2 = -(δ + ε * cos(2π / T * t)) * u[1] - a1 * u[2] + b0 * hist[1] + sin(4π / T * t)
    return @SVector [du1, du2]
end

# Extrakció
params = (δ=..., ε=..., ...)
ldde_sys = extract_SDM_system(mathieu_dde_rhs, params, Val(2); t_test=0.0)
```

### 2. Kézi definíció interfész:

```
using SemiDiscretizationMethod
function createMathieuProblem(δ, ε, b0, a1; T=2π)
    AMx = ProportionalMX(t -> @SMatrix [0.0 1.0; -δ-ε*cos(2π / T * t) -a1])
    τ1 = t -> 2π
    BMx1 = DelayMX(τ1, t -> @SMatrix [0.0 0.0; b0 0.0])
    cVec = Additive(t -> @SVector [0.0, sin(4π / T * t)])
    return LDDEProblem(AMx, [BMx1], cVec)
end
```

### 2.3. Mátrix Előkészítés ("Time Matrix" kiértékelés)

Még az iteráció megkezdése előtt fel kell építeni a rendszermátrixok rácsát.

- Függvény: `build_system_matrices(problem, grid, tableau)`
- Visszatérési érték: Egy adatstruktúra (pl. `Vector{Vector{SMatrix}}`), ahol az `A_matrices[n][i]` adja az  $n$ -edik lépés  $i$ -edik  $c$  idejéhez tartozó mátrixot. Ugyanez a késleltetett  $B$  mátrixokra.
- **Optimalizáció:** Mivel ezeket az értékeket a sajátérték-iteráció során milliószor lekérjük, a memórafolytonos (contiguous) tárolás kulcsfontosságú!

### 2.4. A Lazy Monodrómia Operátor (Forward Substitution)

A Krylov-megoldó nem az egész  $\mathcal{Q}$  leképezést várja, hanem a  $y_p = \Phi y_0$  iterációs lépést. Ezt a blokk alsó háromszög struktúra miatt **előrehelyettesítéssel (forward substitution)** érjük el a Lazy leképezésen belül.

- Függvény: `mul!(y_out::AbstractVector, map::MonodromyMap, x_in::AbstractVector)`
- **A múlt állapotok felépítése:** Az `x_in` a múlt ( $y_0$ ), amelyet a  $T$  és  $\tau$  arányától függően kiegészítünk (padding) vagy azonosítunk az átfedésekkel.
- **Forward Sweep (Előrehelyettesítés):** Egy for-ciklussal végigmegyünk a blokkokon ( $n = 1 \dots p$ ):
  1. A  $\Phi_R$  hatásának kiszámítása: lekérjük az `x_in`-ből a releváns múltbeli és belső állapotokat, és rászorozzuk a  $\mathcal{D}_n$  késleltetett diadikus blokkot.
  2. A  $\Phi_L$  megoldása: Mivel ez tartalmazza a Topológiát és az aktuális rendszermátrixokat, az  $n$ -edik lépés stádiumait lokálisan, iteratíván vagy egy apró `SMatrix` inverzióval kiszámítjuk:  $v_n = \Phi_{L,n}^{-1}(\text{jobb oldal})$ .
  3. Az eredmény továbbvitele a következő  $n + 1$  blokkra.
- **Visszaírás:** A sweep végén a periódus végéhez tartozó  $v_p$  elemeket kicsomagoljuk és beírjuk az `y_out` vektorba.
- *Fontos:* Ezzel a lépésenkénti megoldással sosem tároljuk a sűrű monodrómia mátrixot, csak lokális vektorokon dolgozunk!

### 2.5. Sajátérték Megoldó (KrylovKit)

Mivel a Floquet-szorzók az egyperiódusos átmenet ( $\Phi_L^{-1}\Phi_R$ ) sajátértékei, a `KrylovKit.jl` az ideális eszköz.

- A `mul!` függvényünk pontosan a monodrómia operátor hatását fejezi ki.

```
using KrylovKit
# A KrylovKit automatikusan alkalmazza a mul! függvényt (ami lekezeli a paddingot
```

```
vals, vecs, info = eigsolve(x -> apply_monodromy!(y_temp, MonodromyMap_obj, x), i
```

### 3. A Fejlesztés Lépései (Roadmap a Gemini CLI-hez)

1. **Alapstruktúrák definiálása:** Típusok létrehozása a feladatnak, tábláknak és a rácsnak.
2. **Extrakciós réteg tesztelése:** A DiffEq alapú RHS-ből kinyerni az SMatrixokat adott  $t$  pillanatokban.
3. **Késleltetés és Rendvesztés Modul:** A *Continuous Extension* logikájának megírása. Adott  $\tau$  és időlépés esetén kiszámítani, hogy melyik múltbeli lépés(ek) stádiumait kell milyen súlyokkal lineárisan kombinálni.
4. **A "Time Matrix" generátor:** A lokális  $A_{i,n}$  és  $B_{i,n}$  blokkok előregyártása és optimalizált memóriába töltése.
5. **A Lazy Forward Substitution (  $\mu_1!$  ) megírása:** Ez a módszer lelke. A padding/overlap logika beépítése, majd a blokkonkénti diadikus szorzások és apró lokális invertálások allokációmentes implementálása.
6. **Explicit Sparse építő megírása:** Ellenőrzés gyanánt, azonos eredményt kell adnia, mint a Lazy operátornak (csak lassabban és  $\Phi_L \setminus \Phi_R$  megoldással).
7. **KrylovKit bekötése:** A Floquet-szorzók kiszámítása és validálása a hagyományos SDM csomag eredményeivel (pl. Mathieu-egyenleten).